# Measuring and Gating the Cost of Explainable AI in Fog Computing-based Real-Time Stress Detection from Physiological Signals

Sena Al Hafez, Meryem Eldemery, and Hafiza İrem Türkmen Çilingir
Dept. of Computer Engineering, Yıldız Technical University, Istanbul, Türkiye

***Abstract*—Post-hoc explanations are attached to wearable stress classifiers, but systematic reviews of XAI evaluation exclude run time from their scope, studies that explain such classifiers report accuracy without explanation time, and deployment studies of physiological monitoring measure transport metrics without explanations. We deploy a frozen 49,890-parameter 1D-CNN trained on WESAD as a TCP client–server system, run its server either co-located with the client on a laptop or on a Raspberry Pi 4 fog node, and measure the seven metrics used in fog-computing studies with an open-loop sensor client and drift-corrected clock synchronisation. Without explanation, the server pipeline takes a median of 5.7 ms per 10 s window on the laptop and 10.4 ms on the Pi. A single region-based LIME or SHAP (GradientExplainer) explanation takes a median of 203–784 ms on the laptop (four operating points) and 686–1,474 ms on the Pi (three operating points), i.e. 95–99% of the execution time. The median one-way network latency is at most 4.2 ms, whereas the Pi is 2.4× slower per explanation for LIME with 250 samples and 5.6× slower for SHAP with 50 samples, and every response of the latter exceeds the 1 s real-time bound. On a 43-minute segment in which 20.0% of windows had confidence below 0.80, explaining only those windows reduced the mean execution time 4.5–4.7× and raised the throughput capacity of one worker from 4 to 19 concurrent users. Gating directs explanations to errors only where confidence is calibrated: at the same threshold it explains 26 of 29 misclassified windows of one validation subject but 22 of 184 of another.**

***Index Terms*—explainable AI, fog computing, real-time systems, stress detection, LIME, SHAP, Raspberry Pi**

## I. Introduction

Classifiers for stress detection from wearable physiological signals reach high reported accuracies on public datasets such as WESAD [1], [5], and post-hoc explanations such as LIME [2] and SHAP [3] are attached to them [5], [8]. These explanations are evaluated for quality, not for cost: a systematic review of XAI evaluation explicitly excludes run time from its scope [4], and recent work that explains stress classifiers trained on WESAD reports neither inference nor explanation time [5]. In a real-time monitor, a new 10 s window arrives every 5 s, and the time spent explaining one window is unavailable to the next window and to other users of the same server.

Fog-computing studies of physiological monitoring measure precisely such system-level quantities—latency, queuing delay, execution time, response time, jitter, bandwidth and resource usage—but without explanations [6], [7]. Conversely, measurements of explanation cost are made offline on workstation or server hardware [9], [10], [11]. This paper measures both in one budget, on a resource-constrained fog node of the kind used in those studies. Our contributions are:

1) An end-to-end measurement of explanation cost inside a TCP client–server stress detection pipeline at two placements, using the seven metrics of [6], an open-loop load generator and drift-corrected clock synchronisation (Sec. III).
2) Evidence that explanation accounts for 95–99% of server execution time and that, once it is enabled, placement matters through compute rather than network (Secs. V-A, V-B).
3) A measured characterisation of confidence-gated explanation on the fog node—cost, throughput capacity and the queuing of several users—each matched by a simple analytical model (Secs. V-C, V-D).
4) The calibration condition under which gating directs explanations to errors, and the cost–stability trade-off between explainer operating points (Secs. V-E, V-F).

## II. Related Work

**Fog-based physiological monitoring.** Erzurumluoğlu et al. [6] deploy emotion recognition on DREAMER with a Raspberry Pi 4 fog worker and report the seven transport and resource metrics we adopt. Further fog health systems measure latency, energy, bandwidth and execution time, again without explanations [7], [13]. Conversely, studies that do attach explanations to such systems do not time them (Table I): a fog architecture with LIME and Grad-CAM for facial emotion recognition reports inference throughput but not the cost of producing an explanation [8], a WESAD-trained 1D-CNN with explanations reports no times at all [5], and a LIME-based stress detector on a Jetson Orin Nano reports training times only [14].

**The cost of explanation.** ShaTS [9] states an explicit real-time criterion—an explainer that needs more than 1 s per window is unsuitable for real-time use—and shows KernelSHAP violating it on workstation hardware. KernelSHAP needs seconds to tens of seconds per sample on CPUs [10], [11], whereas amortised estimators such as FastSHAP [11] answer in one forward pass of a trained explainer, at the price of training it. On a Raspberry Pi 4, conditionally triggered SHAP has been reported to add about 195 ms per prediction in a cold-chain monitoring deployment [12].

TABLE I
COMPARISON OF EXISTING STUDIES WITH THE PROPOSED STUDY (✓ = MEASURED/USED; – = ABSENT; GE = GRADIENTEXPLAINER).

| Study | Signals / dataset | Model | XAI | XAI time | Deployment | System metrics | Selective XAI |
|---|---|---|---|---|---|---|---|
| Erzurumluoğlu et al. [6] | EEG+ECG (DREAMER) | RF | – | – | edge, fog (Pi 4), cloud | ✓ (7 metrics) | – |
| Tuli et al. [7] | cardiac data | DL ensemble | – | – | edge, fog, cloud | ✓ | – |
| Gia et al. [13] | ECG | wavelet feat. | – | – | fog gateway | ✓ | – |
| Almadhor et al. [5] | WESAD (+2 sets) | 1D-CNN | ✓ | – | – | – | – |
| El Rashidy et al. [8] | facial | CNN, YOLOv8 | LIME, Grad-CAM | – | fog | frame rate only | – |
| Ghose et al. [14] | HRV (SWELL-KW) | classical ML | LIME | – | edge (Jetson) | training time only | – |
| Franco de la Peña et al. [9] | IIoT (SWaT) | BiLSTM | ShaTS | ✓ | workstation | – | – |
| Warnecke et al. [10] | security data | MLP, CNN, RNN | 6 methods | ✓ | server | – | – |
| Taguta et al. [12] | cold-chain temp. | LSTM-GRU | SHAP | ✓ | fog (Pi 4) | lat., CPU, energy | event-trig. |
| **Proposed** | physiological (WESAD) | 1D-CNN | LIME, SHAP (GE) | ✓ | co-loc., fog (Pi 4) | ✓ (7 metrics) | conf.-gated |

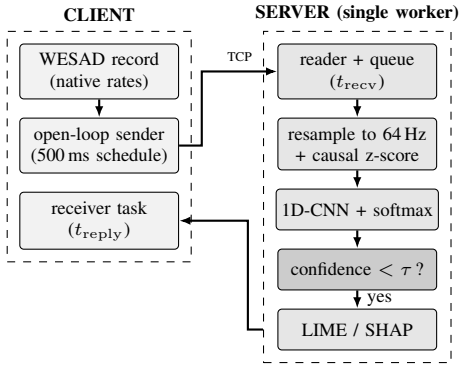



Fig. 1. Measured pipeline. The explanation is produced by the same worker as the prediction, so its cost is part of the measured execution and response times. The server runs either on the client's laptop (co-located) or on the fog node.

**Selective computation.** Gating computation on softmax confidence is established for inference through early exits [15] and confidence cascades [16], and has been proposed for explanations as budgeted selective explanation [17] and as explanations of rejected samples [18]. We do not claim the mechanism. We characterise it end to end in continuous physiological monitoring on constrained hardware, where its benefit depends on calibration [19]. Table I positions this work: no study in the set measures explanation time inside a deployed pipeline together with the transport metrics, or gates explanations on model confidence.

## III. SYSTEM AND MEASUREMENT METHOD

### A. Model and data

The classifier is a temporal 1D-CNN with two convolution blocks ([32, 128] filters, kernel size 9, BatchNorm–ReLU–MaxPool–Dropout), global average pooling and two dense layers (49,890 parameters), trained with focal loss [20] on 14 chest and wrist channels resampled to 64 Hz, in 10 s windows with a 5 s hop. The architecture reaches 0.92 F1 under 15-fold leave-one-subject-out (LOSO) validation. The window and the architecture were fixed in preliminary offline experiments. Window lengths of 1, 30 and 60 s reached F1 scores of 0.816–0.863 there, against the 0.92 of the adopted 10 s configuration, and the deployed network was then selected by a one-factor-at-a-time sweep over depth, filter widths and kernel size that scored every candidate by LOSO F1 together with per-window inference time: the selected two-block network reached a higher sweep F1 than the 105K-parameter three-block baseline (0.931 against 0.927) in two thirds of the baseline's inference time. The deployed checkpoint was trained on 13 subjects; the remaining two, S11 and S14, served as the validation set for checkpoint selection (validation F1 0.939 under offline, full-record normalisation). They were never used for training, but they are not independent test subjects; all measurements below are taken on them. The model is frozen and used unchanged throughout.

### B. Real-time pipeline

The client replays a WESAD record at the sensors' native rates (chest 700 Hz, wrist 4–64 Hz) in 500 ms ticks, each sent as one binary frame of 11,553 bytes. Resampling to 64 Hz is performed by the server and is part of the measured execution time. Offline training normalised every subject with statistics of the complete record, which uses future data; the server instead normalises causally (Welford's algorithm on the data seen so far). Under this constraint the deployed model reaches 0.822 accuracy and 0.694 stress F1 over 1,198 labelled windows of S11 and S14 in an offline replay, and 0.922 accuracy and 0.881 stress F1 over 372 labelled windows of the first 43 minutes of S11 on the fog node.

### C. Transport and measurement

**Single worker with an explicit queue.** A reader task timestamps every frame on arrival ($t_{\mathrm{recv}}$) and enqueues it; a single worker drains the queue ($t_{\mathrm{start}}$, $t_{\mathrm{end}}$). We define latency $= t_{\mathrm{recv}} - t_{\mathrm{send}}$, queuing delay $= t_{\mathrm{start}} - t_{\mathrm{recv}}$, execution time $= t_{\mathrm{end}} - t_{\mathrm{start}}$ and response time $= t_{\mathrm{reply}} - t_{\mathrm{send}}$; these definitions follow [6]. Jitter, the variation of response time in [6], is computed here as the mean absolute difference of consecutive response times. Bandwidth is the number of bytes exchanged per second in both directions, and resource usage is the mean CPU utilisation and peak resident memory (RSS) of the server process. Nagle's algorithm is disabled (`TCP_NODELAY`, the default of Python's asyncio).

**Open-loop load.** A client that waits for each reply before sending the next tick is a *closed* load generator. When an explanation takes longer than one tick, the emulated sensor pauses and the waiting time is omitted from the measurement, which is the coordinated-omission problem of closed-loop

benchmarking [22]. In a check with the 200-sample SHAP explainer, a closed-loop client delayed its sends by 661 ms at the 95th percentile. Our client is open-loop—a sender task emits every tick on schedule, independently of a receiver task—and its send lag on the Pi stayed below 21 ms at the 95th percentile in every single-client run.

**Clocks.** One-way latency requires both clocks: we estimate the offset of the two monotonic clocks NTP-style from minimum-RTT ping exchanges [23]. Because crystal drift turned one-off estimates into negative latencies in long runs, the offset is re-estimated every 60 s and interpolated linearly per packet; the resulting uncertainty (half the median minimum RTT) was at most 1.1 ms.

### *D. Explainers and confidence gating*

**Region-based LIME.** Each $14 \times 640$ window is divided into 20 time segments per channel (280 regions). A masked region is replaced by the channel mean; perturbed samples are weighted with an exponential kernel and fitted by ridge regression. We use 500 samples (*LIME*) and 250 samples (*LIME-cheap*). **SHAP.** SHAP's GradientExplainer (expected gradients [21]) with a fixed 100-window $k$-means background drawn from training subjects in the causal input space, using 200 samples (*SHAP*) and 50 samples (*SHAP-cheap*). These operating points come from a one-factor-at-a-time sweep in the same preliminary experiments over LIME's sample count, segment count and kernel width, and over SHAP's background size, sample count and background-summary method, with every setting scored by faithfulness, monotonicity, sensitivity and latency; within that sweep, the cheap sample counts kept the quality score within 4.9% (LIME) and 1.4% (SHAP) of the best setting while running 2.3× and 8.6× faster than the largest sample counts tried. A cheap point thus runs the same algorithm with a smaller sample count, and its cost scales with that count: on the laptop, halving LIME's samples gives 281 against 543 ms per explanation and quartering SHAP's gives 203 against 784 ms (Table III), with no measurable faithfulness difference between the four points (Sec. V-F). The GradientExplainer is not KernelSHAP, whose cost structure differs [10]. All random seeds are fixed, and explainers are warmed up when the server starts, outside the measurement. The model runs with one PyTorch thread; the numerical libraries used by the explainers (notably BLAS in LIME's ridge regression) keep their default multithreading on both placements.

**Gating.** Let $c$ be the maximum softmax probability of a window. The window is explained if and only if $c < \tau$; $\tau = 0$ never explains and $\tau = 1$ always explains.

## IV. Experimental Setup

**Placements** (Table II). In the *co-located* placement the client and the server run as separate processes on the same laptop and communicate over loopback; this is the lowest-latency placement available, not a wearable device. In the *fog* placement the server runs on a Raspberry Pi 4 Model B with active cooling—the same board model as the fog worker of [6], with 4 GB of memory and a 1.5 GHz part against their 8 GB at 1.8 GHz. Below, "laptop" and "co-located" refer to the same placement.

TABLE II
Placements and software. The same PyTorch release runs on both servers.

| | **Co-located (laptop)** | **Fog** |
|---|---|---|
| Hardware | Intel Core 13th gen. (Raptor Lake), 20 logical cores | Raspberry Pi 4 Model B (rev. 1.1), 4× Cortex-A72, 1.5 GHz |
| Memory | 16 GB | 4 GB |
| OS | Windows 11 | Raspberry Pi OS 64-bit (Debian 13) |
| Software | Python 3.11.9, PyTorch 2.14.1 (CPU), SHAP 0.51.0 | Python 3.13.5, PyTorch 2.14.1 (CPU), SHAP 0.53.0 |
| Link | loopback | LAN; Pi connected by Ethernet |
| CPU gov. | default (Windows) | performance (Table III, Fig. 2); on-demand (Table IV, Sec. V-D) |

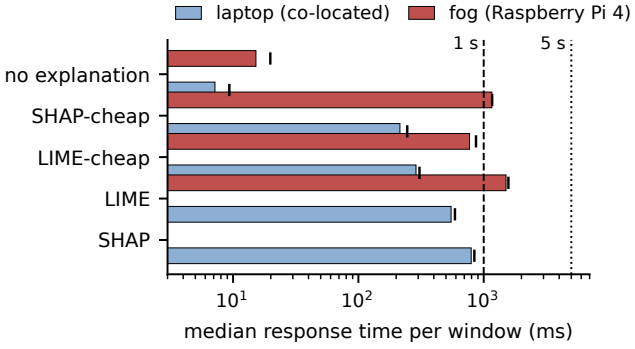



Fig. 2. Median response time per window by explainer and placement, one client (tick = 95th percentile; generated from the same runs as Table III). The 1 s line is the real-time criterion of [9]; the 5 s line is the hop.

**Protocol.** All single-client runs replay subject S11 in real time with causal normalisation; the first 15 windows of every client are discarded as warm-up. Each configuration is measured in one run. The single-client profile of the Pi (Table III) was measured with the CPU governor set to *performance*; the selective and multi-client runs used the default *on-demand* governor and are compared only with always-explain runs under the same governor. Fixed-policy runs (no explanation, or explanation of every window) last 17 minutes (184 windows, the stress-free baseline segment of the record); selective runs last 43 minutes (504 windows, spanning baseline and stress); multi-client runs replay S11 and S14 alternately for 5 minutes. The 10 s window is the length the model was trained with, and the 5 s hop reproduces the 50% window overlap of training. We define the *throughput capacity* of a worker as the hop divided by the mean execution time: the number of clients it can keep up with, regardless of how long an individual response takes.

TABLE III

PLACEMENT COMPARISON, ONE CLIENT, 184 MEASURED WINDOWS PER CONFIGURATION (MS UNLESS STATED; PI WITH THE PERFORMANCE CPU GOVERNOR). LATENCY, EXEC. AND RESP. P50 ARE MEDIANS OVER WINDOWS; XAI IS THE MEDIAN EXPLANATION TIME WITHIN EXECUTION; CPU IS THE MEAN UTILISATION OF THE SERVER PROCESS (100% = ONE CORE); RSS IS ITS PEAK RESIDENT MEMORY. IN EVERY CONFIGURATION THE MEDIAN QUEUING DELAY IS AT MOST 0.1 MS AND BANDWIDTH IS ABOUT 187 KBPS (COLUMNS OMITTED).

| | | Server | | | Client | | | Resources | |
|---|---|---|---|---|---|---|---|---|---|
| **Placement** | **Explainer** | latency | exec. | xai | resp. p50 | resp. p95 | jitter | CPU % | RSS MB |
| Co-located | none | 0.98 | 5.7 | – | 7.2 | 9.3 | 1.4 | 0.5 | 200 |
| (laptop) | SHAP-cheap | 0.61 | 213.4 | 203 | 214.5 | 245.3 | 14.5 | 4.7 | 487 |
| | LIME-cheap | 0.87 | 287.0 | 281 | 288.2 | 307.2 | 11.7 | 6.2 | 439 |
| | LIME | 0.83 | 548.1 | 543 | 549.2 | 589.5 | 20.4 | 11.4 | 451 |
| | SHAP | 0.91 | 793.1 | 784 | 794.7 | 842.1 | 25.8 | 16.2 | 519 |
| Fog | none | 3.03 | 10.4 | – | 15.2 | 19.8 | 2.6 | 0.6 | 245 |
| (Pi 4) | SHAP-cheap | 2.11 | 1,157.1 | 1,146 | 1,161.7 | 1,167.9 | 7.0 | 28.3 | 437 |
| | LIME-cheap | 3.66 | 697.1 | 686 | 770.2 | 868.0 | 78.1 | 33.4 | 521 |
| | LIME | 4.21 | 1,485.0 | 1,474 | 1,507.1 | 1,574.1 | 56.4 | 62.1 | 536 |

## V. RESULTS

### A. Explanation dominates the pipeline

Table III and Fig. 2 profile every configuration. Without explanation, the server spends a median of 5.7 ms per window on the laptop and 10.4 ms on the Pi, well under 1% of the 5 s hop. On the Pi this divides into 2.19 ms for resampling the native streams to 64 Hz, 0.49 ms for causal normalisation, 0.25 ms for tensor preparation, 6.8 ms for inference and 0.14 ms for post-processing (stage medians); the laptop profile has the same composition with 2.5 ms of inference, so every stage of the pipeline fits the real-time budget on both nodes. A single explanation costs a median of 203–784 ms on the laptop and 686–1,474 ms on the Pi and accounts for 95–99% of execution time in every explained configuration. Enabling an explainer raises peak server memory by a factor of 1.8–2.6 (e.g. from 245 to 521 MB on the Pi, 14% of its memory). With one client the median queuing delay is at most 0.1 ms, and bandwidth is about 187 kbps in every configuration, because explanations are computed on the server and not transmitted.

### B. Where to explain: compute, not network

The network contributes little at either placement: the median one-way latency is 0.98 ms over loopback and 3.03 ms to the Pi, and at most 4.2 ms in any single-client configuration. The difference between placements comes from compute. Per explanation, the Pi is 2.4× slower than the laptop for LIME-cheap, 2.7× for LIME, 5.6× for SHAP-cheap. Because the slowdown differs between explainers, their ranking does not transfer across platforms: SHAP-cheap is the fastest explainer on the laptop but slower than LIME-cheap on the Pi. All four operating points meet the real-time criterion of [9] on the laptop, but on the Pi only LIME-cheap does (95th percentile 868.0 ms): every SHAP-cheap response exceeds 1 s (minimum 1,149 ms), and so does the median LIME response (1,507.1 ms). Under explanation, the placement decision is therefore a decision about the compute capability of the node.

Three observations on the Pi qualify these numbers. First, the CPU governor matters for light workloads. Under the default on-demand governor, inference took a median of 14.2 ms in the run without explanation but 7.3 ms in runs where explanations kept the CPU busy. With the performance governor the difference disappears (6.8 versus 7.2 ms), and the execution time without explanation falls from 21.9 to 10.4 ms (2.1×), while explanation time changes by at most 8%. An idle fog node is thus slower than its peak speed for the cheap requests that dominate when explanations are rare. Second, LIME-cheap used on average 2.3 cores of the Pi while busy (mean CPU 33.4% against a busy fraction of 14.5%), because of multithreaded BLAS; a single-threaded LIME would be slower. Third, with LIME-cheap the median gap between response and execution time on the Pi is 33.5 ms, against 4.8 ms without explanation; it spans forward latency, queuing and the reply path, was not isolated further, and is contained in response time and jitter.

TABLE IV

ALWAYS VERSUS SELECTIVE ($\tau = 0.80$) EXPLANATION ON THE PI, BOTH UNDER THE ON-DEMAND GOVERNOR. MEANS ARE REPORTED BECAUSE SELECTIVE RUNS ARE BIMODAL. ALWAYS-RUNS COVER THE FIRST 17 MINUTES OF S11, SELECTIVE RUNS THE FIRST 43; USERS IS THE THROUGHPUT CAPACITY.

| | | | | Exec. (ms) | | Resp. | |
|---|---|---|---|---|---|---|---|
| **Explainer** | **Policy** | **Win.** | **Expl.** | mean | p50 | **mean** | **Users** |
| LIME-cheap | always | 184 | 100% | 772.8 | 755.3 | 805.2 | 6 |
| | selective | 504 | 20.0% | 171.8 | 22.2 | 175.9 | 29 |
| SHAP-cheap | always | 184 | 100% | 1,191.6 | 1,193.3 | 1,195.6 | 4 |
| | selective | 504 | 20.0% | 254.4 | 22.3 | 258.7 | 19 |

### C. Selective explanation on the fog node

With $\tau = 0.80$, 101 of 504 windows (20.0%) fell below the threshold—the same windows for both explainers, because the decision depends only on the model (Table IV). Mean execution time dropped 4.5× for LIME-cheap and 4.7× for SHAP-cheap, and throughput capacity rose from 6 to 29 and from 4 to 19 users. The median window is not explained (median execution 22.3 ms), so the mean is the relevant cost measure. An explained window costs as much as before (per-explanation means within 0.8% of the always-explain runs):

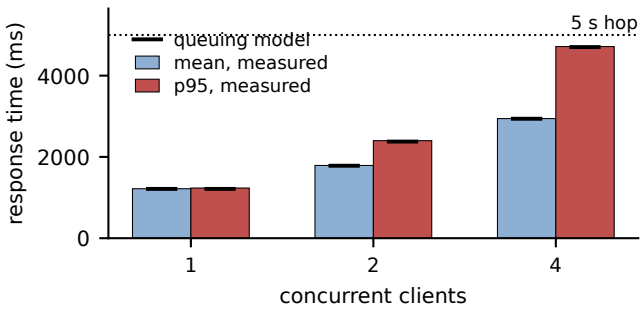



Fig. 3. Pooled response time of one Pi worker serving $N$ synchronised clients, SHAP-cheap on every window (44 windows per client, on-demand governor). Bars are measured; black dashes are the queuing model, $(N{+}1)E/2$ for the mean and $NE$ for the 95th percentile, with $E$ the run's mean execution time.

every explained SHAP-cheap window still exceeds 1 s and every explained LIME-cheap window stays below it, so gating reduces the average load, not the latency of an explained window.

Because both component costs carry over unchanged, the mean execution time of a selective run follows from them: the mean without explanation plus the explanation rate times the mean explanation time gives 173.6 and 256.1 ms against the measured 171.8 and 254.4 ms (about 1% error). Other thresholds can thus be projected from the explanation rate and the per-explanation cost. The rate itself depends on the segment: over the labelled windows of the complete S11 record, the offline replay explains 14.4% at the same threshold.

### D. Several users on one fog worker

One, two and four clients were run against the Pi with SHAP-cheap on every window (44 measured windows per client, pooled over clients; the on-demand governor, as in Table IV). The clients start together, so their windows complete at the same instants and their explanations are served back to back: with $N$ clients and mean execution time $E$ of the run, the response of the $k$-th client served is about $kE$, the mean response is $(N{+}1)E/2$, and the 95th percentile approaches $NE$. The measurements agree with this model within 1.8% (Fig. 3). At four clients the worker is 94% utilised and keeps up with the load, as the throughput capacity of 4 users predicts, but the slowest response approaches the 5 s hop. Throughput capacity is thus not a response-time guarantee: already with two clients the 95th-percentile response is 2,400 ms. Synchronised start is the worst case for this waiting; we did not measure clients with random phases.[1]

### E. The calibration condition

Gating reduces cost on any subject, but it directs explanations to errors only if confidence is calibrated [19]. The model is equally confident on both validation subjects (mean confidence 0.909 and 0.908) but correct on 0.952 and 0.690 of their labelled windows, respectively (Table V). At $\tau = 0.80$ the explanation rates are similar (14.4% and 11.8%), yet 26 of S11's 29 errors are explained but only 22 of S14's 184, because S14 is often wrong with high confidence. Temperature scaling fitted on the other subject made calibration worse for both (ECE 0.256 and 0.276), since the two subjects are miscalibrated in opposite directions. The cost reduction of gating does not depend on calibration, whereas its error coverage does; with two subjects we report this as an observation, which motivates per-user calibration.

[1]These runs used a server version that computed on the event loop, so waiting packets were timestamped late and waiting appeared as latency rather than queuing delay; the client also awaited its periodic clock resynchronisation in the sender task, delaying single sends by up to 3.7 s. Response time, the only metric reported here, is measured on the client and is unaffected by the first issue. Both issues are fixed in the released code.

TABLE V
EXPLANATION RATE AND MISCLASSIFIED WINDOWS THAT ARE EXPLAINED, PER THRESHOLD (LABELLED WINDOWS OF THE COMPLETE RECORDS, CAUSAL INPUT SPACE, UNCALIBRATED SOFTMAX).

| | **S11** (ECE 0.047) | | **S14** (ECE 0.227) | |
|---|---|---|---|---|
| $\tau$ | explained | errors caught | explained | errors caught |
| 0.70 | 10.4% | 18/29 | 4.4% | 11/184 |
| 0.80 | 14.4% | 26/29 | 11.8% | 22/184 |
| 0.90 | 21.4% | 29/29 | 29.3% | 53/184 |

TABLE VI
EXPLAINER OPERATING POINTS: MEDIAN EXPLANATION TIME ON THE LAPTOP AND QUALITY ON CLASS-BALANCED WINDOWS OF S11 (FAITHFULNESS = INSERTION MINUS DELETION AUC $\pm$ STANDARD ERROR, 40 WINDOWS; MONOTONICITY = FRACTION OF NON-DECREASING STEPS OF THE INSERTION CURVE; SENSITIVITY = ATTRIBUTION CHANGE UNDER INPUT NOISE, 10 WINDOWS $\times$ 3 DRAWS, LOWER IS BETTER).

| **Explainer** | **xai (ms)** | **Faithfulness** | **Monoton.** | **Sensitivity** |
|---|---|---|---|---|
| SHAP-cheap | 203 | 0.250 $\pm$0.043 | 0.565 | 0.123 |
| LIME-cheap | 281 | 0.209 $\pm$0.035 | 0.604 | 0.049 |
| LIME | 543 | 0.232 $\pm$0.037 | 0.579 | 0.045 |
| SHAP | 784 | 0.263 $\pm$0.044 | 0.549 | 0.128 |

### F. Explainer quality versus cost

Quality was measured offline on the laptop in the causal input space with an earlier software stack (PyTorch 2.8.0); unlike time, it does not depend on the hardware. No operating point dominates in faithfulness: for every pair the difference is smaller than the sum of the standard errors (Table VI). The explainers differ in stability instead: under small input noise, LIME's attributions change 2.7$\times$ less than those of SHAP. The cheap operating points lose no measurable faithfulness. Since LIME-cheap is also the faster explainer on the Pi (Sec. V-B), it is both the cheaper and the more stable choice on the fog node.

## VI. Discussion and Limitations

**Where the architectural decision moves.** Without explanation, both placements answer well within the hop, and placement changes response time by tens of milliseconds at most—the same order as the differences between the edge, fog and cloud placements of [6] (mean response times 22.5, 43.3 and 73.3 ms). With explanation, two other parameters

dominate: the compute capability of the node, which changes the cost of an explanation by up to 5.6×, and the triggering policy, which changes the mean cost by 4.5–4.7× on our segment. On a Raspberry Pi 4, explaining every window meets the 1 s criterion for a single user with LIME-cheap but not with SHAP-cheap; the throughput capacity of one worker is 4–6 users when every window is explained and 19–29 with gating, while individual explained responses keep their single-user latency only when they do not collide.

**Comparison with the reference system.** The client of [6] also streams raw signals (16 channels at 128 Hz, 32 requests per second; 373.9 kbps against our 187 kbps in binary frames), but its 3 s/1 s windowing and 24-feature random forest make response times not directly comparable. It reports about 20 users per Pi worker without explanations at one decision per second; under our 5 s hop, one worker sustains 4–6 users with every window explained and 19–29 with gating.

**Limitations.** (i) All measurements use the two validation subjects of the deployed checkpoint, and calibration is studied on these two only. (ii) Each configuration is a single run; within-run statistics rest on 44–504 windows, and between-run variance is not reported. In an earlier offline benchmark with 4 alternating runs on the laptop, without network or explanation, run medians varied by 2.9% and 95th percentiles by 12.3%; the effects we draw conclusions from (2.4–5.6× between placements, 4.5–4.7× from gating) are far larger. (iii) The placements share the PyTorch release but differ in processor architecture, operating system, Python and SHAP version; the placement ratio reflects the whole platform, including multithreaded BLAS (Sec. V-B). The Pi runs used a server build that did not log temperature or throttling state; the board was actively cooled. (iv) The gains of gating were measured on one segment with a 20.0% explanation rate; they scale with the rate. (v) One worker per server; a cloud placement was measured earlier with a closed-loop client and a different software stack and is not compared here. (vi) The system replays recorded data at native rates; no user study of explanation usefulness was performed.

## VII. Conclusion

Measured end to end, explanation and not inference is the bottleneck of real-time explainable stress detection: it takes 95–99% of server execution time, and on a Raspberry Pi 4 fog node every 50-sample SHAP explanation takes more than one second. Confidence-gated explanation reduced the mean cost 4.5–4.7× on a segment where 20.0% of windows were explained and raised the throughput capacity of a fog worker by a similar factor, but it directs explanations to errors only where the model is calibrated. The measurement harness, including the open-loop client and drift-corrected clock synchronisation, will be made publicly available.

## References


[1] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing WESAD, a multimodal dataset for wearable stress and affect detection," in *Proc. ACM ICMI*, 2018, pp. 400–408.

[2] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 4765–4774.

[4] M. Nauta *et al.*, "From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI," *ACM Comput. Surv.*, vol. 55, no. 13s, Art. 295, 2023.

[5] A. Almadhor *et al.*, "A cross-domain framework for emotion and stress detection using WESAD, SCIENTISST-MOVE, and DREAMER datasets," *Front. Bioeng. Biotechnol.*, 2025, doi:10.3389/fbioe.2025.1659002.

[6] Ö. F. Erzurumluoğlu, K. Özlem, A. T. Atalay, Ö. Atalay, and G. İnce, "Fog computing-based real-time emotion recognition using physiological signals," in *Proc. 27th Int. Conf. Advanced Communications Technology (ICACT)*, 2025, pp. 442–450.

[7] S. Tuli *et al.*, "HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments," *Future Gener. Comput. Syst.*, vol. 104, pp. 187–200, 2020.

[8] N. El Rashidy, E. Allogmani, E. Hassan, K. Alnowaiser, H. Elmannai, and Z. H. Ali, "Toward real-time emotion recognition in fog computing-based systems: Leveraging interpretable PCA_CNN, YOLO with self-attention mechanism," *Front. Comput. Sci.*, vol. 7, Art. 1714394, 2025.

[9] M. Franco de la Peña, Á. L. Perales Gómez, and L. Fernández Maimó, "ShaTS: A Shapley-based explainability method for time series artificial intelligence models applied to anomaly detection in Industrial Internet of Things," arXiv:2506.01450, 2025.

[10] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in *Proc. IEEE EuroS&P*, 2020, pp. 158–174.

[11] N. Jethani, M. Sudarshan, I. Covert, S.-I. Lee, and R. Ranganath, "FastSHAP: Real-time Shapley value estimation," in *Proc. ICLR*, 2022.

[12] J. Taguta, J. F. Isingizwe Nturambirwe, and C. N. Nyirenda, "Real-world deployment and performance characterisation of fog-based deep learning for cold-chain temperature prediction over LoRaWAN," arXiv:2609.14036, 2026.

[13] T. N. Gia, M. Jiang, A.-M. Rahmani, T. Westerlund, P. Liljeberg, and H. Tenhunen, "Fog computing in healthcare internet of things: A case study on ECG feature extraction," in *Proc. IEEE Int. Conf. Computer and Information Technology (CIT)*, 2015, pp. 356–363.

[14] D. Ghose *et al.*, "Investigating lightweight and interpretable machine learning models for efficient and explainable stress detection," *Front. Digit. Health*, vol. 7, Art. 1523381, 2025.

[15] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. ICPR*, 2016, pp. 2464–2469.

[16] K. Berestizshevsky and G. Even, "Dynamically sacrificing accuracy for reduced computation: Cascaded inference based on softmax confidence," in *Proc. ICANN*, 2019, pp. 306–320.

[17] L. M. Paes, D. Wei, and F. P. Calmon, "Selective explanations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[18] A. Artelt, R. Visser, and B. Hammer, "'I do not know! but why?' — Local model-agnostic example-based explanations of reject," *Neurocomputing*, vol. 558, 2023.

[19] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[20] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, 2017, pp. 2980–2988.

[21] G. Erion, J. D. Janizek, P. Sturmfels, S. M. Lundberg, and S.-I. Lee, "Improving performance of deep learning models with axiomatic attribution priors and expected gradients," *Nature Machine Intelligence*, vol. 3, pp. 620–631, 2021.

[22] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. USENIX NSDI*, 2006.

[23] D. L. Mills, "Internet time synchronization: The network time protocol," *IEEE Trans. Commun.*, vol. 39, no. 10, pp. 1482–1493, 1991.